

Coding Exercise 5

1. (a) Generate $(X_1, X_2, \dots, X_{100}) \in \mathbb{R}^{20}$ such that they are picked i.i.d. $\sim \mathcal{N}([0, 0, \dots, 0], I_{20})$. Generate four numbers $i_1, i_2, i_3, i_4 \in \{1, 2, \dots, 20\}$ uniformly at random without replacement. Also generate four numbers $a, b, c, d \sim \mathcal{N}(0, 0.25)$. Generate $(Y_1, Y_2, \dots, Y_{100}) \in \mathbb{R}$ as given below:

$$Y_k = aX_{ki_1} + bX_{ki_2} + cX_{ki_3} + dX_{ki_4} + n_k,$$

where $n_k \sim \mathcal{N}(0, 0.01)$ for $k = 1, 2, \dots, 100$. Print i_1, i_2, i_3, i_4 .

- (b) Consider the OLS method. Compute and print the coefficient vector $\hat{\beta}^{OLS}$ using the closed form expression. Verify the coefficients using the corresponding command in the *sklearn* library.
- (c) Consider ridge regression. Fix $\alpha = 20$, and compute the coefficient vector $\hat{\beta}^{ridge}$ corresponding to this value of α using the closed form expression for the coefficients. Print $\hat{\beta}^{ridge}$. Verify the coefficients using the corresponding command in the *sklearn* library. Sort the coefficients $\hat{\beta}^{ridge}$ in decreasing order of their absolute value, and then print the indices that correspond to the five highest coefficients.
- (d) Consider lasso method. Repeat what you did for ridge regression but using *sklearn.linear_model.Lasso*.
- (e) Perform a 5-fold cross validation to find the best value of α in part (c) and part (d) from scratch. Compare them with the cross validation errors you obtain using the *GridSearchCV* command. Choose between the following values of α :

$$\alpha \in \{1, 2, 5, 10, 20, 50, 100, 1000, 10000\}.$$

2. (a) Construct a data set consisting of 200 points the same way as in Q1, coding exercise 4. You can use the same code if required.
- (b) Construct the binary tree using classification tree method, solving for the three questions (i.e., which axis, where in the axis, and what classification) from scratch. Use Gini index as the loss function.
- (c) Grow the tree to a maximum depth of 3. The branch can be stopped either if the misclassification is zero, or if the number of points in the rectangle is at most 10. Verify if your answer is the same as that generated by the command *sklearn.tree.DecisionTreeClassifier*.

- (d) Plot the classifying rectangles along with the scatterplot of the generated features. Represent the features having different labels with different colors. Print the training error.
3. Consider the sales of car seats given in *Carseats.csv* file. The first column *Sales* is the dependent variable, and the other columns represent the independent variables. Convert the words in the data as follows:
- Good $\rightarrow$ 1, Medium $\rightarrow$ 0, Bad $\rightarrow$ (-1), Yes $\rightarrow$ 1, No $\rightarrow$ 0.
- (a) Split the data into training and testing data. The training data must contain 80% of the randomly selected data points.
- (b) Construct a binary tree using the command *DecisionTreeRegressor* from *sklearn.tree*. Do not use maximum depth. Split the tree until there are 10 points in each node. Fit the tree in the training data, and then compute the test error in the test data. Print the constructed tree using *plot_tree* from *sklearn.tree*, and also print the test error.
- (c) Build a random forest using the command *RandomForestRegressor* from *sklearn.ensemble*. Fix $B = 100$, and choose four of the ten available parameters in the data set. Print the test error.
4. Consider the training data set with $y = ||x||^2$.
- (a) Pick $x_1, x_2, \dots, x_{10000} \in \mathbb{R}^2$ so that x_i 's are i.i.d. $\sim \text{Unif}[-10, 10]^2$. Generate y_i values using the expression given above.
- (b) Consider the feed-forward neural network with single hidden layer. Choose the right number of nodes in the hidden layer (M) and the right activation function so as to minimize the squared error. You can use a 5-fold cross-validation with $M \in \{5, 10, 20, 50, 100\}$, and activation functions either being "logistic", "tanh", or "relu". Fit the training data using the command *sklearn.neural_network.MLPRegressor*. Print the training error.
- (c) Generate 10,000 test vectors $x_1, x_2, \dots, x_{10000} \in \mathbb{R}^2$ i.i.d. $\sim \text{Unif}[-10, 10]^2$ and test them on the trained networks. Print the test error. Plot $g(x, y) = x^2 + y^2$ in 3D for $x \in [-5, 5]$ and $y \in [-5, 5]$. Plot the prediction by neural networks in the same plot.
5. Consider the problem of detecting spam emails. Download the data from <https://archive.ics.uci.edu/dataset/94/spambase>. You can also import the data in python by using the code given in the tab "import in python".
- (a) Split the data into training and testing data. Let the percentage of test data be 25% of the total data available. Train the data using scikit-learn library with an appropriate value of M and an appropriate activation function, and then test it with the test dataset. Print the training error and the test error.
- (b) Repeat the same with *tensorflow.keras* library.